manually is cumbersome. Then how do you do it? You do it by writing a script that automatically runs through test commands and verifies that the results match the expected behavior specified in the script. For this type of testing, you'll write a script file of all the tests and automate the testing process.

In this chapter, you'll learn about this test automation process for smart contracts. Writing test scripts requires specific knowledge of primitives such as `beforeEach`, `it`, and `describe`, as well and how and when to use them. This chapter illustrates the development of test scripts with three different but familiar smart contracts: counter, ballot, and blind auction. Thus, you are gradually moving from a simple test script to a more complex one. You'll also learn about using the Truffle suite of tools for running the test scripts and verifying that the tests pass (or fail) as you are developing and deploying the smart contracts.

10.1 Importance of testing smart contracts

Those of us who grew with the emergence of integrated chips and system on a chip know the importance of testing. Testing is an essential phase of hardware development. Once a microchip is mass-produced, it is impossible to go back and fix bugs. The design is hardcoded. How about smart contracts, the core logic of our Dapps? Smart contracts are like hardware chips--immutable code. Once deployed, they are final and cannot be updated (unless special provisions or